

FIAP  ONI

05

FRONTEND
MÁGICO:
INTERFACES QUE
ENCANTAM NO
MOBILE

LISTA DE FIGURAS

Figura 1 - Timeline Javascript	9
Figura 2 - Destaques do Vite.....	12
Figura 3 - npm create vite@latest my-react-app -- --template react.....	13
Figura 4 - Interface gráfica do usuário	14
Figura 5 - Interface gráfica do usuário – Aplicativo	14
Figura 6 - Interface gráfica do usuário – Aplicativo (2)	15
Figura 7 - Estrutura do projeto.....	16
Figura 8 - npm install.....	17
Figura 9 - Estrutura do projeto.....	17
Figura 10 - Dependências do projeto	19
Figura 11 - Comportamento do Vite	19
Figura 12 - gitignore	20
Figura 13 - Arquivo HTML	20
Figura 14 - Componentes de Classe.....	21
Figura 15 - Componentes Funcionais	21
Figura 16 - PROPS	22
Figura 17 - Hook useState	22
Figura 18 - useState.....	23
Figura 19 - setState.....	23
Figura 20 - Ciclo de vida dos Componentes	24
Figura 21 - Organização dos Componentes.....	24
Figura 22 - Componente de Classe.....	25
Figura 23 - Componente Funcional.....	25
Figura 24 - Arquivo App.jsx	26
Figura 25 - COMPONENTS/BOASVINDAS.jsx.....	26
Figura 26 - App.jsx	27
Figura 27 - Componentes Funcionais – Contador.....	28
Figura 28 - App.jsx – Contador.....	28
Figura 29 - Componente UsuarioInfo	29
Figura 30 - App.jsx - UsuarioInfo.....	29
Figura 31 - Componente ListaNumeros	29
Figura 32 - App.jsx – ListaNumeros	30
Figura 33 - Componente ContadorSeguro	30
Figura 34 - App.jsx – ContadorSeguro.....	31
Figura 35 - Ciclo de Vida com useEffect	31
Figura 36 - App.jsx – CicloDeVida.....	32
Figura 37 - Componente BotaoEvento	33
Figura 38 - App.jsx – BotaoEvento.....	34
Figura 39 - Componente TextoDinamico.....	34
Figura 40 - App.jsx – TextoDinamico	35
Figura 41 - Componente MensagemLogin	35
Figura 42 - App.jsx – MensagemLogin.....	35
Figura 43 - Componente MensagemUsuario	36
Figura 44 - App.jsx - MensagemUsuario	36
Figura 45 - Componente ListaNomes.....	37
Figura 46 - App.jsx – ListaNomes	37
Figura 47 - Objeto Javascript	37

Figura 48 - CSS.....	38
Figura 49 - Criando arquivo CaixaMensagem.module.css	38
Figura 50 - App.jsx - CaixaMensagem	39
Figura 51 - Componente BotaoEstilizado	39
Figura 52 - App.jsx – BotaoEstilizado.....	40
Figura 53 - STYLED COMPONENTS	40
Figura 54 - ThemeProvider.....	40
Figura 55 - AlertaTailwind.....	41
Figura 56 - App.jsx – AlertaTailwind	41
Figura 57 - AlertaTailwind para criar layouts responsivos	42
Figura 58 - Classes Tailwind	42
Figura 59 - Componente hook useState.....	43
Figura 60 - App.jsx – hook useState	43
Figura 61 - Componente Relógio	45
Figura 62 - App.jsx – Componente Relógio.....	45
Figura 63 - Limites do useContext.....	47
Figura 64 - Componente PerfilUsuario	47
Figura 65 - App.jsx – PerfilUsuario.....	48

LISTA DE QUADROS

Quadro 1 - Comparação entre abordagens.....	42
---	----

ENCANTO

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Comando para criar um novo projeto React com Vite	13
Comando de prompt 2 - Instalação das dependências do projeto	16
Comando de prompt 3 - Instalação da biblioteca de estilos	39
Comando de prompt 4 - Instala, configura e inicializa os estilos do projeto	41
Comando de prompt 5 - Importa, organiza e aplica os estilos do projeto.....	41

ENCANTO

SUMÁRIO

1 FRONTEND MÁGICO: INTERFACES QUE ENCANTAM NO MOBILE.....	8
1.1 A Evolução do Desenvolvimento Web	8
1.2 O Ecossistema Javascript Atual	8
1.3 Bibliotecas <i>versus</i> Frameworks.....	9
1.4 Por que React se Tornou Popular?	10
2 CONFIGURAÇÃO DO AMBIENTE COM VITE E ESTRUTURA DE PROJETO	11
2.1 O que é Vite e por que utilizá-lo?	11
2.2 Instalação e Configuração Inicial.....	12
2.2.1 Pré-requisitos	12
2.2.2 Passo a passo para criação do projeto React com Vite	13
2.2.3 Entrar na pasta do projeto e abrir no VS Code.....	14
2.2.4 Instalar as dependências do projeto.....	15
2.3 Estrutura de Diretórios de um Projeto React + Vite.....	17
2.4 Entendendo os Arquivos de Configuração	18
2.4.1 package.json	18
2.4.2 vite.config.js.....	19
2.4.3 .gitignore.....	19
2.4.4 index.html	20
3 CONCEITOS BÁSICOS DO REACT.....	21
3.1 Componentes: Funcionais <i>versus</i> Classe	21
3.2 Props: Passagem de Dados entre Componentes	22
3.3 Estado: Gerenciamento de Dados Mutáveis	22
3.3.1 Estado em componentes funcionais.....	22
3.3.2 Estado com Objetos e Arrays.....	23
3.3.3 Atualizações baseadas em estado anterior	23
3.4 Ciclo de Vida dos Componentes	23
3.5 Implementando os Conceitos no Projeto my-react-app.....	24
3.5.1 Criando componentes funcionais	25
3.5.2 Props: Passando dados entre componentes.....	26
3.5.3 Estado: Armazenando informações em um componente	27
3.5.4 Funcionamento de um componente funcional.....	27
3.5.5 Criando componentes com Objetos e Arrays	28
3.5.6 Atualizando componentes com gerenciamento de estado	30
3.5.7 Ciclo de vida com useEffect	31
4 GERENCIAMENTO DE EVENTOS E DOM VIRTUAL	33
4.1 Como o React Lida com Eventos	33
4.2 O Conceito de DOM Virtual e Reconciliação.....	34
4.3 Renderização Condicional.....	35
4.3.1 Ternário e operador lógico	35
4.3.2 Uso de variáveis e componentes	36
4.4 Listas e Chaves no React.....	36
4.4.1 Chaves com IDs únicos <i>versus</i> índices.....	37
5 ESTILIZAÇÃO EM APLICAÇÕES REACT.....	38
5.1 CSS Modules	38
5.2 Styled Components	39

5.2.1 Estendendo estilos com styled components.....	40
5.2.2 Theming global - src/theme.js	40
5.3 TailwindCSS.....	41
5.3.1 Responsividade e componentização	42
5.4 Comparação entre Abordagens	42
6 HOOKS FUNDAMENTAIS	43
6.1 useState.....	43
6.2 useEffect.....	44
6.2.1 Para que serve o useEffect?	44
6.2.2 Quando o useEffect é executado?	44
6.2.3 Exemplo prático.....	45
6.3 useContext.....	45
6.3.1 O que é o useContext?.....	46
6.3.2 Como funciona o uso de contexto?	46
6.3.3 Quando usar useContext?.....	46
6.3.4 Limites do useContext.....	47
REFERÊNCIAS.....	49

1 FRONTEND MÁGICO: INTERFACES QUE ENCANTAM NO MOBILE

1.1 A Evolução do Desenvolvimento Web

O desenvolvimento web passou por transformações significativas desde o seu surgimento. No início dos anos 90, as páginas eram estáticas, compostas basicamente por HTML e, posteriormente, por alguns estilos em CSS. A interatividade era limitada e, quando existia, era implementada de forma rudimentar com JavaScript.

Com o avanço da tecnologia e o aumento da demanda por experiências mais ricas, as aplicações web evoluíram de simples páginas de conteúdo para aplicações completas que operam diretamente no navegador. Entretanto, essa mudança trouxe novos desafios: como gerenciar a complexidade crescente? Como garantir performance em aplicações robustas? Como oferecer uma boa experiência ao desenvolvedor?

A resposta veio com a adoção de bibliotecas e frameworks JavaScript, que introduziram uma nova abordagem na construção de interfaces ricas, interativas e escaláveis. Com isso, entramos na era das **Single Page Applications (SPAs)**, onde a navegação é fluida e o carregamento da página acontece apenas uma vez, otimizando a experiência do usuário.

1.2 O Ecossistema Javascript Atual

O ecossistema JavaScript se expandiu de forma exponencial na última década, oferecendo soluções para quase todas as necessidades do desenvolvimento front-end:

- **Bibliotecas de UI:** React, Vue, Svelte;
- **Frameworks completos:** Angular, Next.js, Nuxt.js;
- **Gerenciadores de estado:** Redux, Zustand, Recoil;
- **Ferramentas de build:** Webpack, Rollup, Vite, Parcel;
- **Frameworks CSS:** Bootstrap, TailwindCSS, Material UI;

- **Ferramentas de testes:** Jest, Testing Library, Cypress.

Essa diversidade oferece liberdade de escolha, mas também pode gerar confusão entre pessoas iniciantes. O importante é entender que cada ferramenta possui um conjunto de objetivos e pontos fortes. O domínio do ecossistema exige compreensão dos fundamentos e das necessidades específicas de cada projeto.

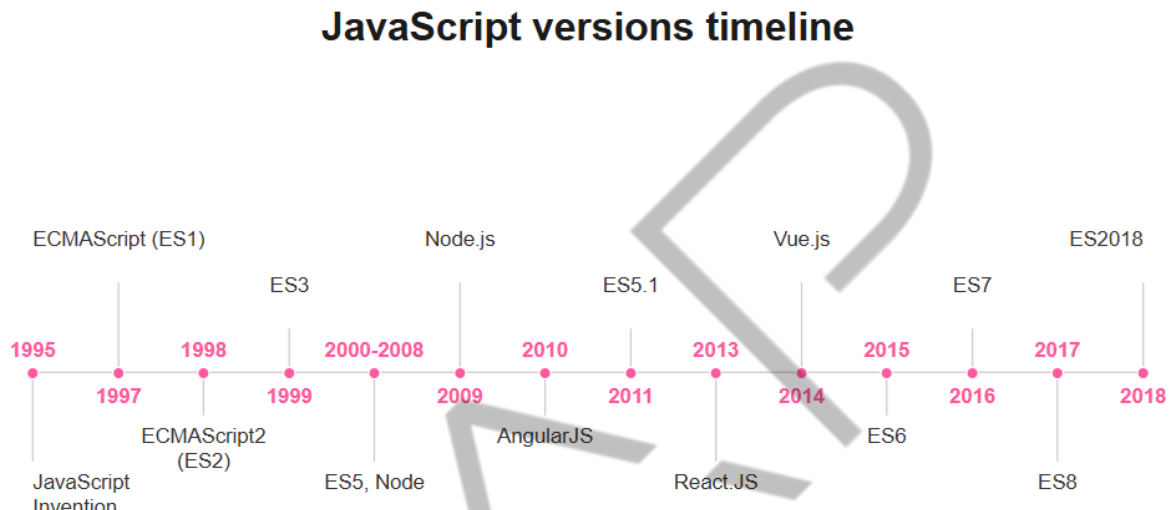


Figura 1 - Timeline Javascript
Fonte: Medium (2023), adaptado por FIAP (2025)

1.3 Bibliotecas *versus* Frameworks

Antes de aprofundarmos no React, é essencial compreender a diferença entre bibliotecas e frameworks:

- **Bibliotecas** são conjuntos de funções e componentes que você utiliza conforme sua necessidade. Elas oferecem flexibilidade e liberdade arquitetural. O React, por exemplo, é uma biblioteca voltada exclusivamente para construção de interfaces, sem impor regras rígidas de estrutura.
- **Frameworks**, por outro lado, são estruturas completas e opinativas que definem como o seu projeto deve ser organizado. Angular é um exemplo: ele fornece soluções integradas para roteamento, formulários, testes e muito mais.

A diferença conceitual é que **você chama uma biblioteca**, enquanto **um framework chama seu código**. Com bibliotecas, há mais autonomia, mas também

mais responsabilidade. Com frameworks, o desenvolvimento pode se tornar mais rápido, porém menos flexível.

1.4 Por que React se Tornou Popular?

Desde seu lançamento pelo Facebook em 2013, o React se consolidou como uma das bibliotecas mais utilizadas no mundo para desenvolvimento front-end. Entre os principais fatores de sua adoção destacam-se:

- **Modelo declarativo:** permite descrever a UI de forma clara e direta, tornando o código mais legível e previsível.
- **Componentização:** incentiva o uso de componentes reutilizáveis, facilitando a manutenção e a evolução de sistemas complexos.
- **Portabilidade de conceitos:** os fundamentos aprendidos com React são aplicáveis em diversas plataformas como React Native (mobile) e Electron (desktop).
- **Virtual DOM:** a manipulação eficiente do DOM com reconciliação garante desempenho superior mesmo em aplicações grandes.
- **Ecossistema expansível:** uma gama de bibliotecas e ferramentas complementam o React, como React Router, Redux, Zustand, Styled Components e mais.
- **Adoção por grandes empresas:** Facebook, Instagram, Netflix, Airbnb e outros gigantes adotaram o React, o que fortaleceu ainda mais sua comunidade e credibilidade.

2 CONFIGURAÇÃO DO AMBIENTE COM VITE E ESTRUTURA DE PROJETO

2.1 O que é Vite e por que utilizá-lo?

O Vite é uma ferramenta de build moderna, criada para proporcionar uma experiência de desenvolvimento mais rápida e eficiente, especialmente em projetos JavaScript modernos como aplicações React. O nome “Vite” vem do francês, que significa “rápido” — e essa é justamente sua principal vantagem.

Motivos para usar o Vite:

- **Inicialização instantânea:** ao invés de empacotar toda a aplicação na inicialização, como o Webpack, o Vite utiliza **ES Modules nativos do navegador** e só carrega os arquivos conforme a necessidade.
- **Atualizações instantâneas (HMR):** Hot Module Replacement é extremamente rápido, o que acelera o ciclo de desenvolvimento.
- **Configuração zero:** basta um único comando para iniciar um projeto React com Vite. Não é necessário configurar Babel, Webpack ou outros transpilers manualmente.
- Suporte a TypeScript, JSX, CSS Modules, PostCSS e outras ferramentas modernas por padrão.

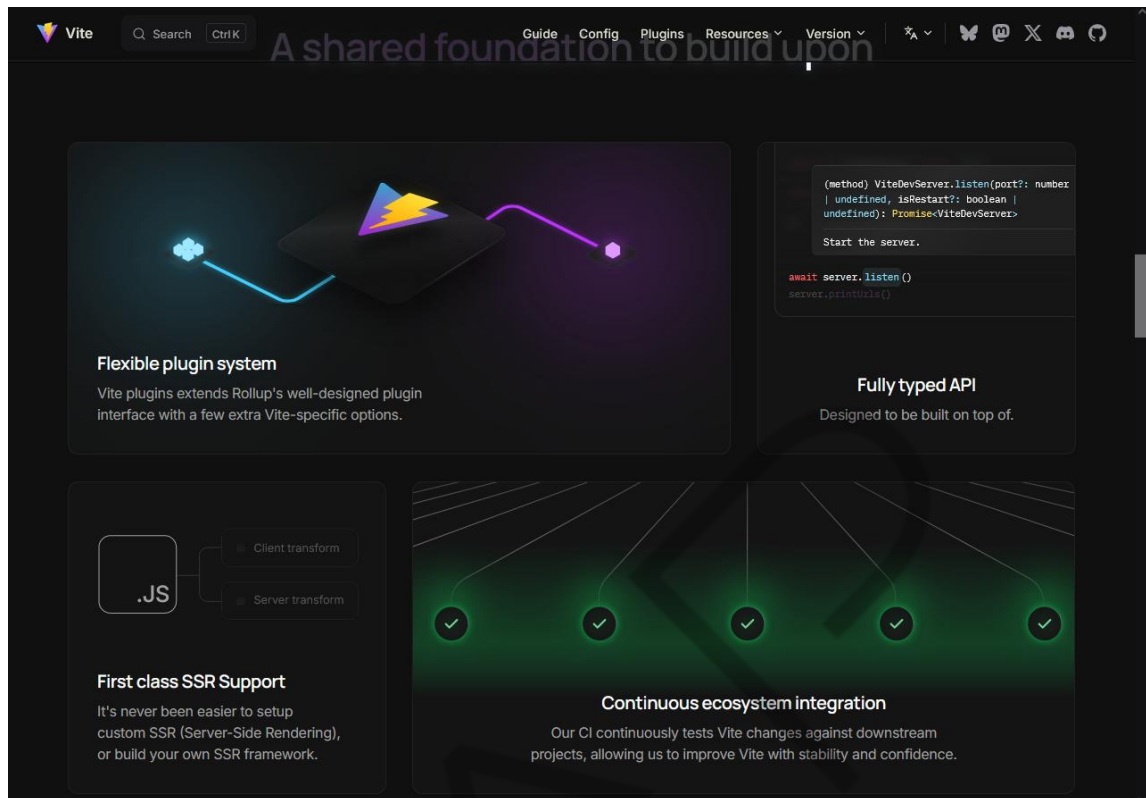


Figura 2 - Destaques do Vite
Fonte: Vite (2025)

2.2 Instalação e Configuração Inicial

Nesta seção, explicaremos o passo a passo para a criação de um novo projeto **React com Vite**, utilizando um terminal moderno e compatível com comandos Unix. Ao longo do processo, recomendamos o uso do **Git Bash** — um terminal leve e prático que simula um ambiente Unix em sistemas Windows — ideal para trabalhar com ferramentas modernas do ecossistema JavaScript.

Importante: não recomendamos o uso do terminal nativo cmd do Windows, pois ele pode apresentar problemas de compatibilidade na execução de comandos, especialmente durante a instalação de pacotes e execução do Vite.

2.2.1 Pré-requisitos

Antes de iniciar, verifique se os seguintes itens estão corretamente instalados em sua máquina:

- Node.js (versão 16 ou superior);

- NPM (instalado automaticamente com o Node.js);
- Git Bash ou PowerShell como terminal de execução (recomendamos **Git Bash**, incluído na instalação padrão do Git);
- Editor de código, como o Visual Studio Code (VS Code).

2.2.2 Passo a passo para criação do projeto React com Vite

Abra o **Git Bash** (ou PowerShell) e navegue até a pasta onde deseja criar o projeto.

Em seguida, execute o seguinte comando:

```
npm create vite@latest my-react-app -- --template react
```

Comando de prompt 1 - Comando para criar um novo projeto React com Vite
Fonte: Elaborado pelo autor (2025)

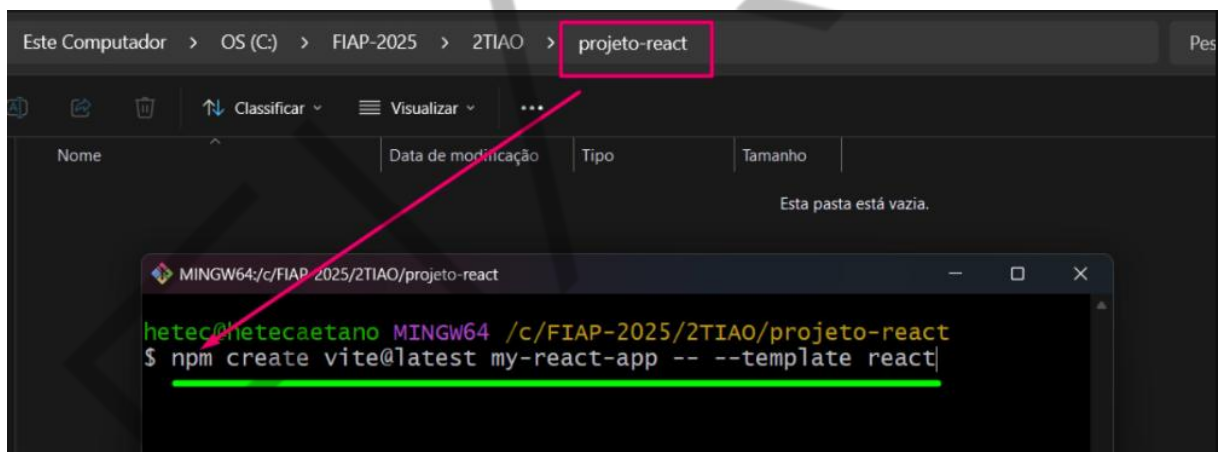


Figura 3 - npm create vite@latest my-react-app -- --template react
Fonte: Elaborado pelo autor (2025)

Este comando criará uma nova pasta chamada my-react-app, contendo a estrutura inicial de um projeto React com Vite.

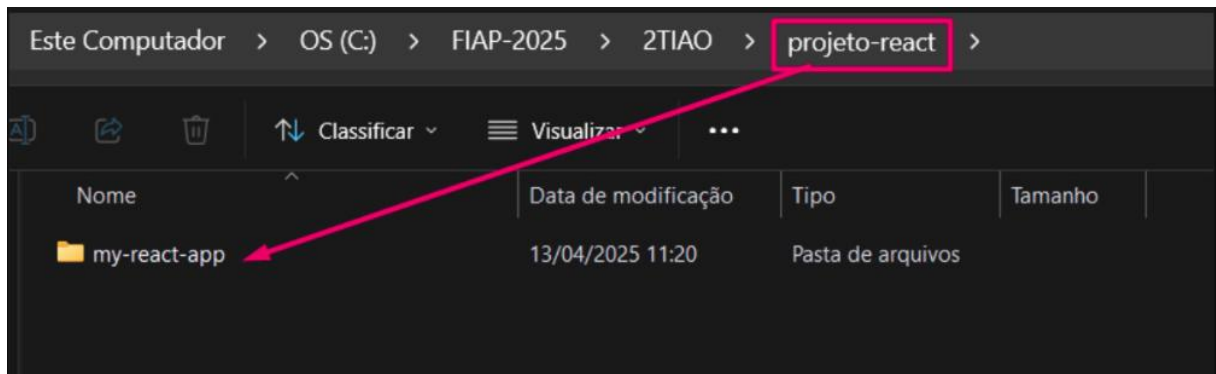


Figura 4 - Interface gráfica do usuário
Fonte: Elaborado pelo autor (2025)

2.2.3 Entrar na pasta do projeto e abrir no VS Code

Após a criação do projeto, será necessário abrir o diretório do projeto no Visual Studio Code para facilitar a navegação e edição dos arquivos.

Passo a passo:

1. **Abra o Visual Studio Code** normalmente em sua máquina;
2. No menu superior, clique em **"File" > "Open Folder..."** (ou **"Arquivo" > "Abrir Pasta..."**, se estiver em português);

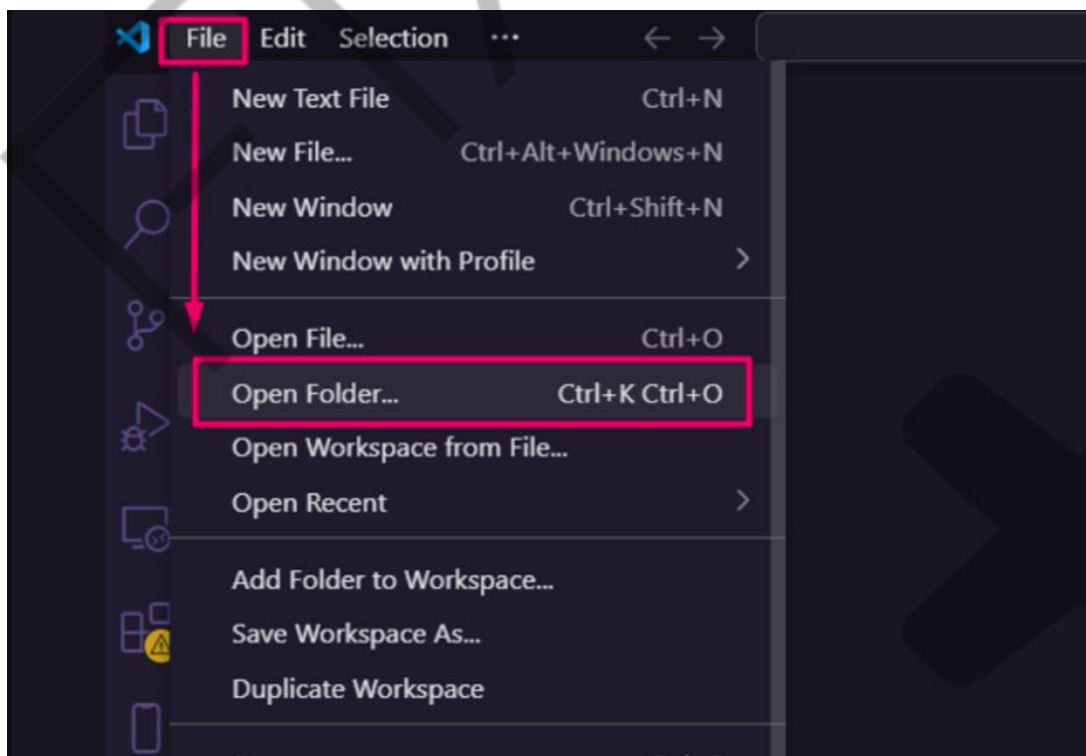


Figura 5 - Interface gráfica do usuário – Aplicativo
Fonte: Elaborado pelo autor (2025)

3. Navegue até o local onde você criou o projeto e selecione a pasta my-react-app;
4. Clique em "**Selecionar pasta**" para carregar o projeto no VS Code.

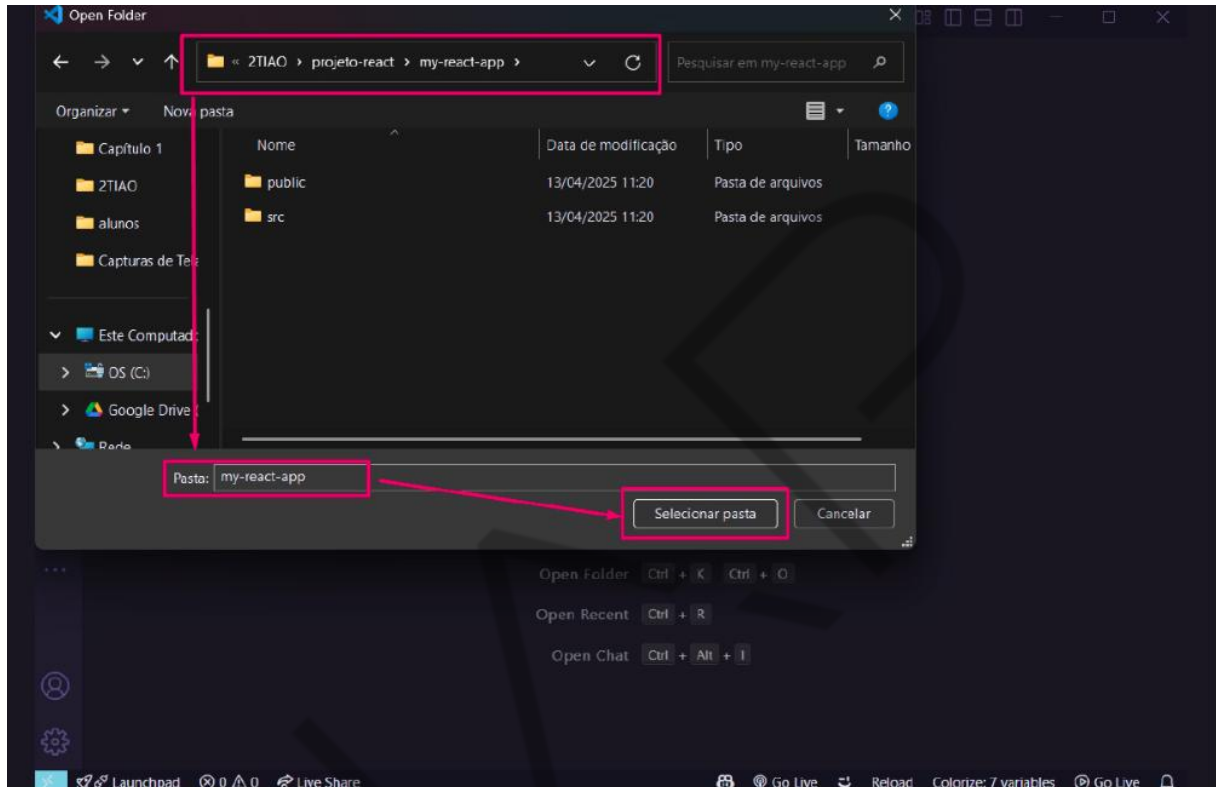


Figura 6 - Interface gráfica do usuário – Aplicativo (2)
Fonte: Elaborado pelo autor (2025)

2.2.4 Instalar as dependências do projeto

Após abrir a pasta my-react-app no **Visual Studio Code**, você estará visualizando pela primeira vez a estrutura do projeto React com Vite.

No lado esquerdo da tela (na **barra lateral do VS Code**), você verá diversas pastas e arquivos como **src**, **public**, **package.json**, entre outros. Esses elementos compõem a base do seu projeto. O arquivo package.json, por exemplo, lista todas as bibliotecas que o projeto necessita para funcionar.

Antes de continuar, você precisará abrir o terminal para instalar essas dependências.

Passo a passo para abrir o terminal:

1. No menu superior do VS Code, clique em "**Terminal**" > "**New Terminal**" (ou "**Terminal**" > "**Novo Terminal**", em português);
2. O terminal será exibido na parte inferior do editor, já posicionado dentro da pasta do projeto my-react-app.
 - o Verifique se a linha de comando mostra algo como:

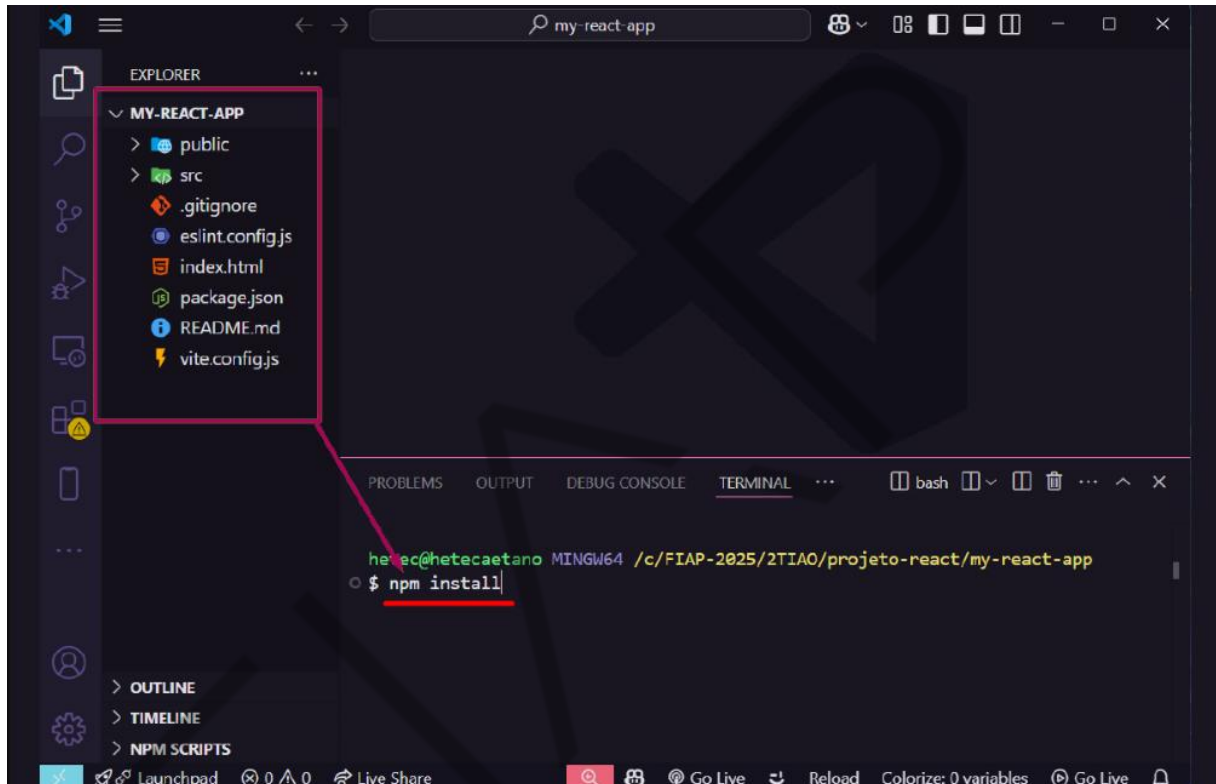


Figura 7 - Estrutura do projeto
Fonte: Elaborado pelo autor (2025)

Com o terminal aberto, execute o seguinte comando:

npm install

Comando de prompt 2 - Instalação das dependências do projeto
Fonte: Elaborado pelo autor (2025)

Esse comando lerá o arquivo package.json e instalará automaticamente todas as bibliotecas necessárias para que o projeto funcione, incluindo:

- **React:** biblioteca principal para construção da interface;
- **Vite:** ferramenta moderna de build e desenvolvimento;
- Outras dependências auxiliares essenciais ao projeto.

Aguarde até que o processo de instalação seja concluído. Ele pode demorar alguns instantes, dependendo da velocidade da sua Internet.

```
hetec@hetecaetano MINGW64 /c/FIAP-2025/2TIA0/projeto-react/my-react-app
$ npm install

added 150 packages, and audited 151 packages in 12s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Figura 8 - npm install
Fonte: Elaborado pelo autor (2025)

2.3 Estrutura de Diretórios de um Projeto React + Vite

Ao executar o comando de criação do projeto com Vite, uma estrutura padrão de arquivos e pastas é automaticamente gerada. A seguir está a estrutura resultante, com a descrição de cada item.

1	my-react-app/	
2	├─ node_modules/	→ Bibliotecas e dependências instaladas via npm
3	├─ public/	→ Arquivos estáticos públicos acessíveis diretamente (ex: favicon, imagens)
4	├─ src/	→ Código-fonte da aplicação React
5	├─ .gitignore	→ Arquivos e pastas a serem ignorados pelo Git
6	├─ eslint.config.js	→ Configuração do ESLint (validação de código e boas práticas)
7	├─ index.html	→ Arquivo HTML principal da aplicação
8	├─ package-lock.json	→ Registro exato das versões das dependências instaladas
9	├─ package.json	→ Informações do projeto e lista de dependências/scripts
10	├─ README.md	→ Instruções ou documentação inicial do projeto
11	└─ vite.config.js	→ Arquivo de configuração do Vite

Figura 9 - Estrutura do projeto
Fonte: Elaborado pelo autor (2025)

Explicação dos principais diretórios e arquivos:

- **node_modules/**: contém todos os pacotes instalados via npm. Essa pasta não deve ser editada manualmente.
- **public/**: é o local para arquivos públicos que não passam pelo sistema de build do Vite.
- **src/**: diretório onde você escreverá os componentes React e o restante do código da aplicação.

- **.gitignore:** arquivo que define quais arquivos/pastas não devem ser versionados no Git (como node_modules/).
- **eslint.config.js:** arquivo de configuração do ESLint, usado para garantir qualidade e padronização do código.
- **index.html:** HTML base da aplicação. O React será injetado no elemento `<div id="root">`.
- **package.json:** arquivo principal de manifesto do projeto, contendo nome, versão, scripts e dependências.
- **vite.config.js:** configuração do Vite, onde é possível ajustar plugins, aliases e otimizações.

2.4 Entendendo os Arquivos de Configuração

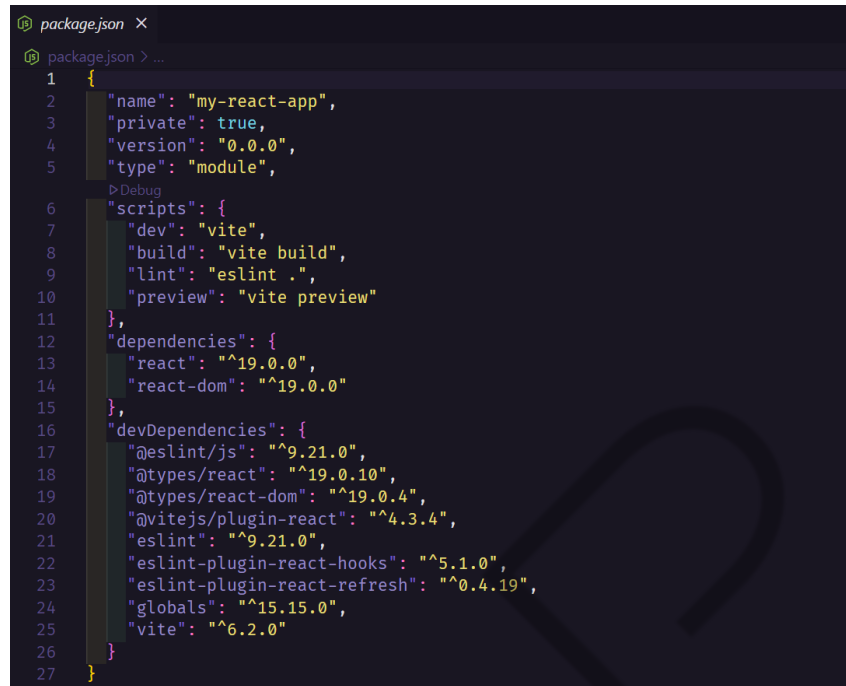
Neste tópico, vamos analisar os principais arquivos de configuração que controlam o comportamento do projeto:

2.4.1 package.json

Esse é o arquivo de manifesto do projeto. Ele contém:

- O nome do projeto e versão;
- Scripts que podem ser executados (npm run dev, npm run build etc.);
- Dependências e devDependencies;
- Configurações do projeto, como engines e repositórios.

Frontend Mágico: Interfaces que Encantam no Mobile

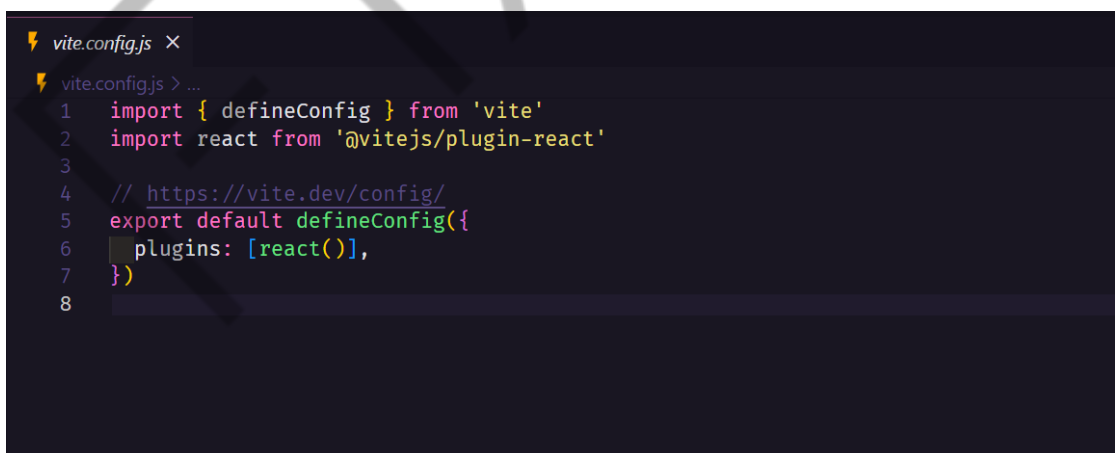


```
package.json X
package.json > ...
1 {
2   "name": "my-react-app",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "lint": "eslint .",
10    "preview": "vite preview"
11  },
12  "dependencies": {
13    "react": "^19.0.0",
14    "react-dom": "^19.0.0"
15  },
16  "devDependencies": {
17    "@eslint/js": "^9.21.0",
18    "@types/react": "^19.0.10",
19    "@types/react-dom": "^19.0.4",
20    "@vitejs/plugin-react": "^4.3.4",
21    "eslint": "^9.21.0",
22    "eslint-plugin-react-hooks": "^5.1.0",
23    "eslint-plugin-react-refresh": "^0.4.19",
24    "globals": "^15.15.0",
25    "vite": "^6.2.0"
26  }
27 }
```

Figura 10 - Dependências do projeto
Fonte: Elaborado pelo autor (2025)

2.4.2 vite.config.js

Trata-se de um arquivo opcional, porém gerado por padrão, serve para personalizar o comportamento do Vite — como aliases, plugins, proxy, entre outros.

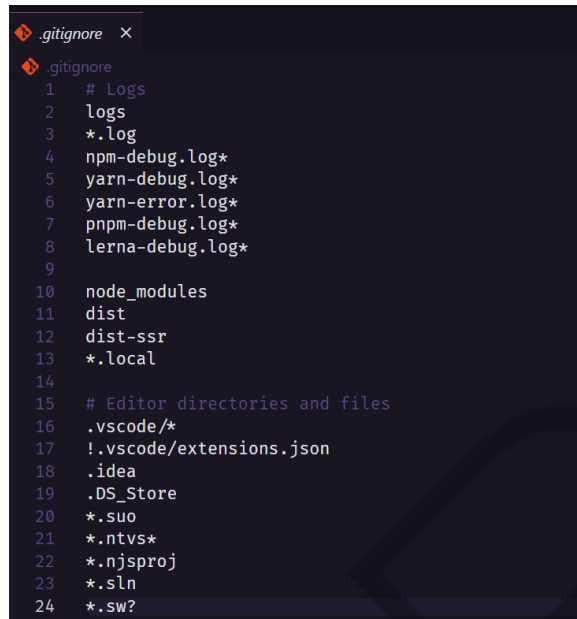


```
vite.config.js X
vite.config.js > ...
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7 })
8
```

Figura 11 - Comportamento do Vite
Fonte: Elaborado pelo autor (2025)

2.4.3 .gitignore

É a lista de arquivos e pastas que não devem ser rastreados pelo Git. Por exemplo:

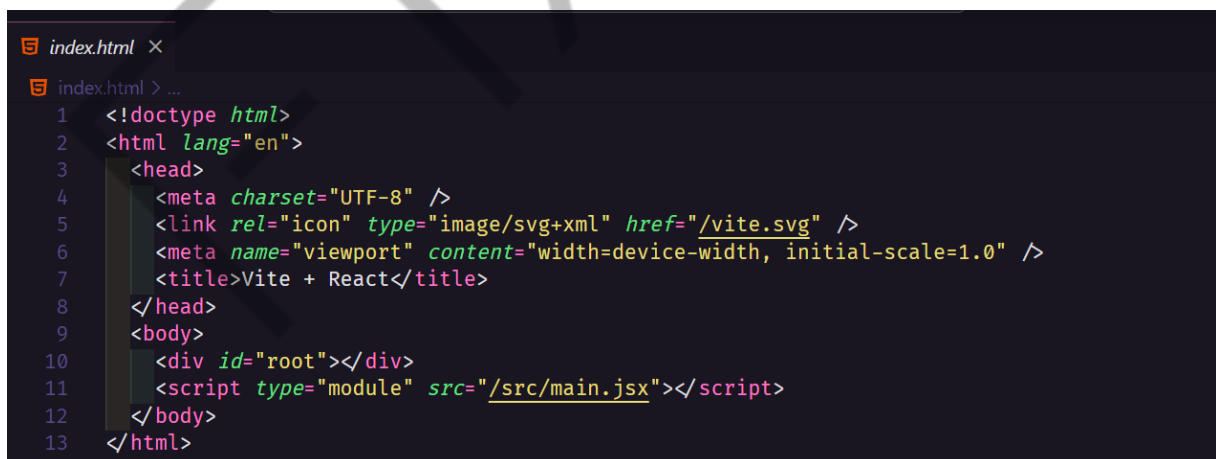


```
.gitignore
1 # Logs
2 logs
3 *.log
4 npm-debug.log*
5 yarn-debug.log*
6 yarn-error.log*
7 pnpm-debug.log*
8 lerna-debug.log*
9
10 node_modules
11 dist
12 dist-ssr
13 *.local
14
15 # Editor directories and files
16 .vscode/*
17 !.vscode/extensions.json
18 .idea
19 .DS_Store
20 *.suo
21 *.ntvs*
22 *.njsproj
23 *.sln
24 *.sw?
```

Figura 12 - gitignore
Fonte: Elaborado pelo autor (2025)

2.4.4 index.html

É o arquivo HTML principal da aplicação. Ao contrário do Create React App, o index.html em projetos Vite é totalmente acessível e editável. O React será injetado no elemento com id="root".



```
index.html
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>Vite + React</title>
8   </head>
9   <body>
10    <div id="root"></div>
11    <script type="module" src="/src/main.jsx"></script>
12  </body>
13 </html>
```

Figura 13 - Arquivo HTML
Fonte: Elaborado pelo autor (2025)

3 CONCEITOS BÁSICOS DO REACT

3.1 Componentes: Funcionais *versus* Classe

O React é baseado no conceito de **componentes**, que são blocos reutilizáveis de código responsáveis por renderizar partes da interface. Existem dois tipos principais de componentes:

- **Componentes de Classe:** antes do React 16.8, a forma mais comum de lidar com o estado e o ciclo de vida era por meio de classes. Um componente de classe precisa estender `React.Component` e implementar um método **`render()`**.

```
1 import React, { Component } from 'react';
2 class Saudacao extends Component {
3   render() {
4     return <h1>Olá, {this.props.nome}!</h1>;
5   }
6 }
```

Figura 14 - Componentes de Classe
Fonte: Elaborado pelo autor (2025)

- **Componentes Funcionais:** são funções JavaScript que retornam elementos React. Desde a introdução dos **Hooks**, os componentes funcionais passaram a ter a mesma capacidade que os de classe, mas com um código muito mais simples.

```
1 function Saudacao(props) {
2   return <h1>Olá, {props.nome}!</h1>;
3 }
```

Figura 15 - Componentes Funcionais
Fonte: Elaborado pelo autor (2025)

Hoje, a recomendação oficial da equipe do React é utilizar **componentes funcionais com Hooks**.

3.2 Props: Passagem de Dados entre Componentes

As **props** (propriedades) permitem que componentes recebam dados externos e os utilizem na renderização. São imutáveis e fornecidas de um componente pai para um componente filho.

```
1 function BoasVindas(props) {  
2   return <p>Bem-vindo, {props.usuario}!</p>;  
3 }  
4 function App() {  
5   return <BoasVindas usuario="Maria" />;  
6 }
```

Figura 16 - PROPS
Fonte: Elaborado pelo autor (2025)

Observação: as props são fundamentais para tornar componentes reutilizáveis e dinâmicos.

3.3 Estado: Gerenciamento de Dados Mutáveis

Diferente das props, o **estado** (state) representa dados internos do componente que podem mudar com o tempo. O estado é mutável e pode ser alterado por interações do usuário, chamadas assíncronas etc.

3.3.1 Estado em componentes funcionais

Com a introdução do **Hook useState**, é possível usar o estado em componentes funcionais.

```
1 import { useState } from 'react';  
2 function Contador() {  
3   const [contador, setContador] = useState(0);  
4   return (  
5     <div>  
6       <p>Você clicou {contador} vezes</p>  
7       <button onClick={() => setContador(contador + 1)}>Incrementar</button>  
8     </div>  
9   );  
10 }
```

Figura 17 - Hook useState
Fonte: Elaborado pelo autor (2025)

3.3.2 Estado com Objetos e Arrays

O **useState** também pode armazenar objetos ou arrays. Ao modificar, é importante usar o operador spread (...) para manter a imutabilidade.

```
1 const [usuario, setUsuario] = useState({ nome: 'João', idade: 25 });
2 setUsuario({ ...usuario, idade: 26 });
3 const [lista, setLista] = useState([1, 2, 3]);
4 setLista([...lista, 4]);
```

Figura 18 - useState
Fonte: Elaborado pelo autor (2025)

3.3.3 Atualizações baseadas em estado anterior

Para atualizações que dependem do valor anterior do estado, recomenda-se usar uma função como argumento do **setState**:

```
1 setContador(prev => prev + 1);
```

Figura 19 - setState
Fonte: Elaborado pelo autor (2025)

Isso evita problemas com atualizações assíncronas.

3.4 Ciclo de Vida dos Componentes

O ciclo de vida define como um componente é criado, atualizado e destruído.

Nos componentes de classe, utilizamos métodos como:

- **componentDidMount()** — chamado após o componente ser montado na tela;
- **componentDidUpdate()** — chamado após uma atualização de estado ou props;
- **componentWillUnmount()** — chamado antes do componente ser destruído.

Nos componentes funcionais, usamos o **Hook useEffect** para representar todos esses estágios:

```
1 import { useEffect } from 'react';
2 function Exemplo() {
3   useEffect(() => {
4     console.log('Componente montado');
5     return () => {
6       console.log('Componente será desmontado');
7     };
8   }, []);
9   return <div>Componente com ciclo de vida</div>;
10 }
```

Figura 20 - Ciclo de vida dos Componentes
Fonte: Elaborado pelo autor (2025)

- O segundo parâmetro [] indica que o efeito ocorre apenas uma vez, como **componentDidMount**.
- A função de retorno representa o comportamento de **componentWillUnmount**.

3.5 Implementando os Conceitos no Projeto my-react-app

Agora que estudamos os **conceitos básicos do React** — incluindo componentes funcionais e de classe, props, estado e ciclo de vida —, vamos **colocar tudo em prática** dentro do projeto my-react-app que criamos anteriormente.

O objetivo é que cada conceito tenha um **exemplo funcional no código**, organizado de forma clara, para facilitar a compreensão e entendimento.

Organização dos Componentes

Antes de começarmos, crie uma nova pasta chamada components dentro do diretório src/ do seu projeto:

```
1 my-react-app/
2 |— src/
3 |   |— components/ ← Aqui ficarão todos os componentes criados neste capítulo
4 |   |— App.jsx
5 |   |— ...
```

Figura 21 - Organização dos Componentes
Fonte: Elaborado pelo autor (2025)

Explorando o Projeto: Testando Componentes no App.jsx

No React, o arquivo App.jsx funciona como a porta de entrada principal da aplicação — é nele que você decide o que será exibido na tela.

Cada vez que criamos um novo componente, precisamos importá-lo no **App.jsx** e utilizá-lo dentro do retorno da função App(). Isso permite que ele seja renderizado pelo React.

Importante: os exemplos a seguir são independentes. Para testá-los, você deve comentar ou remover os exemplos anteriores do App.jsx e manter apenas o que está sendo estudado. Isso evita conflitos e permite que você veja o funcionamento de cada componente isoladamente.

3.5.1 Criando componentes funcionais

Vamos criar dois componentes que exibem a mesma mensagem, mas com sintaxes diferentes. A ideia é **comparar a forma antiga (classe)** com a **forma moderna (função)**.

a) Criar componente de classe em **src/components/SaudacaoClasse.jsx**

```
1 import React, { Component } from 'react';
2 class SaudacaoClasse extends Component {
3   render() {
4     return <h1>Olá, {this.props.nome}!</h1>;
5   }
6 }
7 export default SaudacaoClasse;
```

Figura 22 - Componente de Classe
Fonte: Elaborado pelo autor (2025)

b) Criar componente funcional em **src/components/SaudacaoFuncional.jsx**

```
1 function SaudacaoFuncional(props) {
2   return <h1>Olá, {props.nome}!</h1>;
3 }
4 export default SaudacaoFuncional;
```

Figura 23 - Componente Funcional
Fonte: Elaborado pelo autor (2025)

c) Testar ambos no **src/App.jsx**

```
1 import SaudacaoClasse from './components/SaudacaoClasse';
2 import SaudacaoFuncional from './components/SaudacaoFuncional';
3 function App() {
4   return (
5     <div>
6       <SaudacaoClasse nome="Turma React - Classe" />
7       <SaudacaoFuncional nome="Turma React - Funcional" />
8     </div>
9   );
10 }
11 export default App;
```

Figura 24 - Arquivo App.jsx
Fonte: Elaborado pelo autor (2025)

Observação: este exemplo mostra dois componentes diferentes sendo renderizados juntos no App.jsx. Isso é útil para comparação lado a lado.

3.5.2 Props: Passando dados entre componentes

No exemplo anterior, utilizamos props implicitamente ao passar nome="...". Agora, vamos aprofundar esse conceito com um componente específico.

a) Criar componente em **src/components/BoasVindas.jsx**

```
1 function BoasVindas(props) {
2   return <p>Bem-vindo, {props.usuario}!</p>;
3 }
4 export default BoasVindas;
```

Figura 25 - COMPONENTS/BOASVINDAS.jsx
Fonte: Elaborado pelo autor (2025)

b) Testar o componente no **src/App.jsx**

Atenção: antes de continuar, **substitua o conteúdo do App.jsx** pelo exemplo a seguir ou comente os anteriores.

```
1 import BoasVindas from './components/BoasVindas';
2 function App() {
3   return (
4     <div>
5       <BoasVindas usuario="Maria" />
6     </div>
7   );
8 }
9 export default App;
```

Figura 26 - App.jsx
Fonte: Elaborado pelo autor (2025)

Dica: embora o React não exija que apenas um componente seja exibido por vez, é recomendado que você execute apenas um de cada vez no **App.jsx** durante os estudos. Isso ajudará você a entender o que cada parte do código está fazendo.

3.5.3 Estado: Armazenando informações em um componente

O **estado (ou state)** permite armazenar informações dentro de um componente e atualizá-las ao longo do tempo, principalmente em resposta a interações do usuário.

Diferente das **props**, que vêm de fora, o **estado é controlado internamente** pelo próprio componente.

Importante: para cada exemplo a seguir, **substitua temporariamente o conteúdo do App.jsx** para testar o funcionamento de forma isolada. Isso ajuda a focar no comportamento de cada componente.

3.5.4 Funcionamento de um componente funcional

a) Criar o componente **Contador** em **src/components/Contador.jsx**

```
1 import { useState } from 'react';
2 function Contador() {
3   const [contador, setContador] = useState(0);
4   return (
5     <div>
6       <p>Você clicou {contador} vezes</p>
7       <button onClick={() => setContador(contador + 1)}>Incrementar</button>
8     </div>
9   );
10 }
11 export default Contador;
```

Figura 27 - Componentes Funcionais – Contador
Fonte: Elaborado pelo autor (2025)

b) Testar **Contador** no **App.jsx**

```
1 import Contador from './components/Contador';
2 function App() {
3   return (
4     <div>
5       <Contador />
6     </div>
7   );
8 }
9 export default App;
```

Figura 28 - App.jsx – Contador
Fonte: Elaborado pelo autor (2025)

Sempre que o botão é clicado, o número aumenta. Esse valor é armazenado no estado interno (useState) e o React automaticamente atualiza a interface.

3.5.5 Criando componentes com Objetos e Arrays

a) Criar componente **UsuariInfo** em **src/components/UsuariInfo.jsx**

Frontend Mágico: Interfaces que Encantam no Mobile

```
1 import { useState } from 'react';
2 function UsuarioInfo() {
3   const [usuario, setUsuario] = useState({ nome: 'João', idade: 25 });
4   const aumentarIdade = () => {
5     setUsuario({ ...usuario, idade: usuario.idade + 1 });
6   };
7   return (
8     <div>
9       <p>{usuario.nome} tem {usuario.idade} anos</p>
10      <button onClick={aumentarIdade}>Aumentar idade</button>
11    </div>
12  );
13 }
14 export default UsuarioInfo;
```

Figura 29 - Componente UsuarioInfo
Fonte: Elaborado pelo autor (2025)

b) Testar **usuarioinfo** no **App.js**

```
1 import UsuarioInfo from './components/UsuarioInfo';
2 function App() {
3   return (
4     <div>
5       <UsuarioInfo />
6     </div>
7   );
8 }
9 export default App;
```

Figura 30 - App.jsx - UsuarioInfo
Fonte: Elaborado pelo autor (2025)

c) Criar componente **listanumeros** em **src/components/listanumeros.jsx**

```
1 import { useState } from 'react';
2 function ListaNumeros() {
3   const [lista, setLista] = useState([1, 2, 3]);
4   const adicionarNumero = () => {
5     setLista([...lista, lista.length + 1]);
6   };
7   return (
8     <div>
9       <ul>
10        {lista.map((num, idx) => <li key={idx}>{num}</li>)}
11      </ul>
12      <button onClick={adicionarNumero}>Adicionar número</button>
13    </div>
14  );
15 }
16 export default ListaNumeros;
```

Figura 31 - Componente ListaNumeros
Fonte: Elaborado pelo autor (2025)

d) Testar **ListaNumeros** no **App.jsx**

```
1 import ListaNumeros from './components/ListaNumeros';
2 function App() {
3   return (
4     <div>
5       <ListaNumeros />
6     </div>
7   );
8 }
9 export default App;
```

Figura 32 - App.jsx – ListaNumeros
Fonte: Elaborado pelo autor (2025)

Dica: sempre que trabalhar com objetos ou arrays no estado, use o **operador spread (...)** para garantir que você está criando uma nova estrutura e mantendo a imutabilidade dos dados.

3.5.6 Atualizando componentes com gerenciamento de estado

a) Criar componente **ContadorSeguro** em **src/components/ContadorSeguro.jsx**

```
1 import { useState } from 'react';
2 function ContadorSeguro() {
3   const [contador, setContador] = useState(0);
4   return (
5     <div>
6       <p>Contador: {contador}</p>
7       <button onClick={() => setContador(prev => prev + 1)}>Incrementar com segurança</button>
8     </div>
9   );
10 }
11 export default ContadorSeguro;
```

Figura 33 - Componente ContadorSeguro
Fonte: Elaborado pelo autor (2025)

b) Testar **ContadorSeguro** no **App.jsx**

```
1 import ContadorSeguro from './components/ContadorSeguro';
2 function App() {
3   return (
4     <div>
5       <ContadorSeguro />
6     </div>
7   );
8 }
9 export default App;
```

Figura 34 - App.jsx – ContadorSeguro
Fonte: Elaborado pelo autor (2025)

Por que isso é importante? Quando o novo valor depende do valor anterior, é **recomendado** passar uma função para o `setState`. Isso evita conflitos quando múltiplas atualizações ocorrem rapidamente.

3.5.7 Ciclo de vida com `useEffect`

Componentes React possuem **ciclos de vida**, ou seja, etapas pelas quais passam:

- **Montagem:** quando aparece na tela;
- **Atualização:** quando o estado ou as props mudam;
- **Desmontagem:** quando o componente é removido da tela.

No React moderno (funcional), usamos o **Hook `useEffect`** para controlar essas fases.

a) Criar componente **CicloDeVida** em **src/components/CicloDeVida.jsx**

```
1 import { useEffect } from 'react';
2 function CicloDeVida() {
3   useEffect(() => {
4     console.log('Componente montado');
5     return () => {
6       console.log('Componente será desmontado');
7     };
8   }, []);
9   return <div>Veja o console do navegador para o ciclo de vida</div>;
10 }
11 export default CicloDeVida;
```

Figura 35 - Ciclo de Vida com `useEffect`
Fonte: Elaborado pelo autor (2025)

b) Testar **CicloDeVida** no **App.jsx**

```
1 import CicloDeVida from './components/CicloDeVida';
2 function App() {
3   return (
4     <div>
5       <CicloDeVida />
6     </div>
7   );
8 }
9 export default App;
```

Figura 36 - App.jsx – CicloDeVida
Fonte: Elaborado pelo autor (2025)

Dica: abra o console do navegador clicando em “F12 > aba Console” para ver as mensagens de montagem e desmontagem. Isso ajuda a entender **quando o React executa ações automáticas** com base no ciclo de vida.

Com isso, teremos:

- Uma aplicação prática de cada conceito apresentado neste capítulo;
- Clareza sobre como e onde utilizar o App.jsx;
- Entendimento progressivo para os fundamentos do React.

4 GERENCIAMENTO DE EVENTOS E DOM VIRTUAL

Agora, iremos explorar como o React lida com eventos de interação — como cliques, entradas de texto e formulários —, além de entender o funcionamento interno do **DOM Virtual**, incluindo suas otimizações de renderização.

Cada seção será demonstrada na prática com componentes separados para facilitar a aprendizagem.

4.1 Como o React Lida com Eventos

No React, o gerenciamento de eventos é feito de forma semelhante ao HTML tradicional, mas com algumas diferenças:

- Os nomes dos eventos usam **camelCase** (exemplo: `onClick`);
- Os manipuladores de eventos são passados como funções (`onClick={() => ...}`).

a) Criar componente **BotaoEvento** em `src/components/BotaoEvento.jsx`

```
1 function BotaoEvento() {  
2   function handleClick() {  
3     alert('Você clicou no botão!');  
4   }  
5   return (  
6     <button onClick={handleClick}>Clique aqui</button>  
7   );  
8 }  
9 export default BotaoEvento;
```

Figura 37 - Componente BotaoEvento
Fonte: Elaborado pelo autor (2025)

b) Testar **BotaoEvento** no **App.js**

```
1 import BotaoEvento from './components/BotaoEvento';
2 function App() {
3   return (
4     <div>
5       <BotaoEvento />
6     </div>
7   );
8 }
9 export default App;
```

Figura 38 - App.jsx – BotaoEvento
Fonte: Elaborado pelo autor (2025)

4.2 O Conceito de DOM Virtual e Reconciliação

O React utiliza o **DOM Virtual** (Virtual DOM), uma representação em memória da árvore de elementos da interface. Isso permite atualizações eficientes por meio de um processo chamado **reconciliação**:

- O React compara o DOM atual com o DOM anterior;
- Atualiza somente os elementos que realmente mudaram.

Esse processo garante performance e economia de recursos.

a) Criar componente **TextoDinamico** em **src/components/TextoDinamico.jsx**

```
1 import { useState } from 'react';
2 function TextoDinamico() {
3   const [texto, setTexto] = useState('Texto inicial');
4   return (
5     <div>
6       <p>{texto}</p>
7       <input
8         type="text"
9         value={texto}
10        onChange={e => setTexto(e.target.value)}
11      />
12     </div>
13   );
14 }
15 export default TextoDinamico;
```

Figura 39 - Componente TextoDinamico
Fonte: Elaborado pelo autor (2025)

b) Testar **TextoDinamico** no **App.jsx**

```
1 import TextoDinamico from './components/TextoDinamico';
2 function App() {
3   return (
4     <div>
5       <TextoDinamico />
6     </div>
7   );
8 }
9 export default App;
```

Figura 40 - App.jsx – TextoDinamico
Fonte: Elaborado pelo autor (2025)

4.3 Renderização Condicional

O React permite alterar o que será exibido na tela com base em condições. Essa técnica é chamada de **renderização condicional**.

4.3.1 Ternário e operador lógico

a) Criar componente **MensagemLogin** em **src/components/MensagemLogin.jsx**

```
1 function MensagemLogin({ logado }) {
2   return (
3     <div>
4       {logado ? <p>Bem-vindo de volta!</p> : <p>Você precisa fazer login.</p>}
5     </div>
6   );
7 }
8 export default MensagemLogin;
```

Figura 41 - Componente MensagemLogin
Fonte: Elaborado pelo autor (2025)

b) Testar no **App.jsx**

```
1 import MensagemLogin from './components/MensagemLogin';
2 function App() {
3   return (
4     <div>
5       <MensagemLogin logado={true} />
6     </div>
7   );
8 }
9 export default App;
```

Figura 42 - App.jsx – MensagemLogin
Fonte: Elaborado pelo autor (2025)

4.3.2 Uso de variáveis e componentes

a) Criar componente **MensagemUsuario** em **src/components/MensagemUsuario.jsx**

```
1 function MensagemUsuario({ usuario }) {  
2   let conteudo;  
3   if (usuario) {  
4     conteudo = <p>Olá, {usuario}!</p>;  
5   } else {  
6     conteudo = <p>Nenhum usuário logado.</p>;  
7   }  
8   return <div>{conteudo}</div>;  
9 }  
10 export default MensagemUsuario;
```

Figura 43 - Componente MensagemUsuario
Fonte: Elaborado pelo autor (2025)

b) Testar no **App.jsx**

```
1 import MensagemUsuario from './components/MensagemUsuario';  
2 function App() {  
3   return (  
4     <div>  
5       <MensagemUsuario usuario="Carlos" />  
6     </div>  
7   );  
8 }  
9 export default App;
```

Figura 44 - App.jsx - MensagemUsuario
Fonte: Elaborado pelo autor (2025)

4.4 Listas e Chaves no React

Ao renderizar listas de elementos, é necessário fornecer uma **chave (key)** **única** para cada item. Isso permite que o React identifique quais itens foram alterados, adicionados ou removidos.

a) Criar componente **ListaNomes** em **src/components/ListaNomes.jsx**

```
1 function ListaNomes() {
2   const nomes = ['Ana', 'Bruno', 'Carlos'];
3   return (
4     <ul>
5       {nomes.map((nome, index) => (
6         <li key={index}>{nome}</li>
7       ))}
8     </ul>
9   );
10 }
11 export default ListaNomes;
```

Figura 45 - Componente ListaNomes

Fonte: Elaborado pelo autor (2025)

b) Testar **ListaNomes** no **App.jsx**

```
1 import ListaNomes from './components/ListaNomes';
2 function App() {
3   return (
4     <div>
5       <ListaNomes />
6     </div>
7   );
8 }
9 export default App;
```

Figura 46 - App.jsx – ListaNomes

Fonte: Elaborado pelo autor (2025)

4.4.1 Chaves com IDs únicos *versus* índices

Se cada item da lista tiver um ID único (exemplo: vindo de um banco de dados), prefira usá-lo como chave:

```
1 const usuarios = [
2   { id: 1, nome: 'Ana' },
3   { id: 2, nome: 'Bruno' },
4   { id: 3, nome: 'Carlos' }
5 ];
6 <ul>
7   {usuarios.map((usuario) => (
8     <li key={usuario.id}>{usuario.nome}</li>
9   ))}
10 </ul>
```

Figura 47 - Objeto Javascript

Fonte: Elaborado pelo autor (2025)

Dica: evite utilizar o index como chave quando os itens podem mudar de ordem. Isso pode causar erros sutis na atualização do DOM Virtual.

5 ESTILIZAÇÃO EM APLICAÇÕES REACT

Agora, vamos explorar as diferentes abordagens de estilização em aplicações React modernas. Cada técnica será aplicada com exemplos práticos usando o projeto **my-react-app**, para que você compreenda as vantagens, limitações e melhores cenários de uso de cada uma.

5.1 CSS Modules

Os **CSS Modules** permitem escrever estilos em arquivos `.module.css` que são aplicados somente ao componente onde foram importados, evitando conflitos globais.

a) Criar componente **CaixaMensagem** em **src/components/CaixaMensagem.jsx**

```
1 import styles from './CaixaMensagem.module.css';
2 function CaixaMensagem() {
3   return <div className={styles.mensagem}>Olá, este é um componente estilizado com CSS Module!</div>;
4 }
5 export default CaixaMensagem;
```

Figura 48 - CSS
Fonte: Elaborado pelo autor (2025)

b) Criar arquivo **CaixaMensagem.module.css** na mesma pasta.

```
1 .mensagem {
2   background-color: #f0f0f0;
3   padding: 16px;
4   border: 1px solid #ccc;
5   border-radius: 8px;
6   font-weight: bold;
7 }
```

Figura 49 - Criando arquivo CaixaMensagem.module.css
Fonte: Elaborado pelo autor (2025)

c) Testar no **App.jsx**

```
1 import CaixaMensagem from './components/CaixaMensagem';
2 function App() {
3   return <CaixaMensagem />;
4 }
5 export default App;
```

Figura 50 - App.jsx - CaixaMensagem
Fonte: Elaborado pelo autor (2025)

5.2 Styled Components

A biblioteca **styled-components** permite escrever CSS diretamente dentro do JavaScript, com escopo automático e sintaxe moderna.

Instale com o comando:

```
npm install styled-components
```

Comando de prompt 3 - Instalação da biblioteca de estilos
Fonte: Elaborado pelo autor (2025)

a) Criar componente **BotaoEstilizado** em
src/components/BotaoEstilizado.jsx

```
1 import styled from 'styled-components';
2 const Botao = styled.button`
3   background-color: #007bff;
4   color: white;
5   padding: 10px 20px;
6   border: none;
7   border-radius: 5px;
8   cursor: pointer;
9   &:hover {
10     background-color: #0056b3;
11   }
12 `;
13 function BotaoEstilizado() {
14   return <Botao>Clique aqui</Botao>;
15 }
16 export default BotaoEstilizado;
```

Figura 51 - Componente BotaoEstilizado
Fonte: Elaborado pelo autor (2025)

b) Testar no **App.jsx**

```
1 import BotaoEstilizado from './components/BotaoEstilizado';
2 function App() {
3   return <BotaoEstilizado />;
4 }
5 export default App;
```

Figura 52 - App.jsx – BotaoEstilizado
Fonte: Elaborado pelo autor (2025)

5.2.1 Estendendo estilos com styled components

É possível herdar e estender estilos de um componente para outro.

```
1 const BotaoPrimario = styled.button`
2   background-color: green;
3   color: white;
4 `;
5 const BotaoSecundario = styled(BotaoPrimario)`
6   background-color: gray;
7 `;
```

Figura 53 - STYLED COMPONENTS
Fonte: Elaborado pelo autor (2025)

5.2.2 Theming global - src/theme.js

Também é possível aplicar temas globais com ThemeProvider.

```
1 export const theme = {
2   cores: {
3     primario: '#007bff',
4     texto: '#333'
5   }
6 };
7 // src/App.jsx
8 import { ThemeProvider } from 'styled-components';
9 import { theme } from './theme';
10 function App() {
11   return (
12     <ThemeProvider theme={theme}>
13       {/* componentes aqui */}
14     </ThemeProvider>
15   );
16 }
```

Figura 54 - ThemeProvider
Fonte: Elaborado pelo autor (2025)

5.3 TailwindCSS

TailwindCSS é um framework utilitário que permite aplicar estilos diretamente nas classes dos elementos HTML ou JSX.

Instalação:

```
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

Comando de prompt 4 - Instala, configura e inicializa os estilos do projeto
Fonte: Elaborado pelo autor (2025)

Configure o arquivo **tailwind.config.js** e adicione ao **index.css**.

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

Comando de prompt 5 - Importa, organiza e aplica os estilos do projeto
Fonte: Elaborado pelo autor (2025)

Criar componente **AlertaTailwind** em **src/components/AlertaTailwind.jsx**.

```
1 function AlertaTailwind() {  
2   return (  
3     <div className="bg-yellow-200 border-1-4 border-yellow-500 text-yellow-700 p-4">  
4       Alerta: Esta é uma mensagem importante!  
5     </div>  
6   );  
7 }  
8 export default AlertaTailwind;
```

Figura 55 - AlertaTailwind
Fonte: Elaborado pelo autor (2025)

Testar no **App.jsx**

```
1 import AlertaTailwind from './components/AlertaTailwind';  
2 function App() {  
3   return <AlertaTailwind />;  
4 }  
5 export default App;
```

Figura 56 - App.jsx – AlertaTailwind
Fonte: Elaborado pelo autor (2025)

5.3.1 Responsividade e componentização

Tailwind permite criar layouts responsivos com classes utilitárias.

```
1 <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
2   { /* cards aqui */ }
3 </div>
```

Figura 57 - AlerteTailwind para criar layouts responsivos
Fonte: Elaborado pelo autor (2025)

Também é possível criar componentes reaproveitáveis com classes Tailwind.

```
1 const Botao = ({ children }) => (
2   <button className="bg-blue-500 text-white py-2 px-4 rounded hover:bg-blue-700">
3     {children}
4   </button>
5 );
```

Figura 58 - Classes Tailwind
Fonte: Elaborado pelo autor (2025)

5.4 Comparação entre Abordagens

Abordagem	Escopo Local	Estilo no JS	Temas Globais	Performance
CSS Modules	Sim	Não	Limitado	Alta
Styled Components	Sim	Sim	Sim	Média
TailwindCSS	Simulado	Não	Sim (via config.)	Alta

Quadro 1 - Comparação entre abordagens
Fonte: Elaborado pelo autor (2025)

Recomendações:

- **Iniciantes:** comece com CSS Modules;
- **Projetos dinâmicos:** use styled-components pela flexibilidade;
- **Grandes times e produtividade:** o TailwindCSS se destaca pela velocidade e consistência.

6 HOOKS FUNDAMENTAIS

Hooks são funções especiais do React que permitem utilizar recursos como **estado**, **efeitos colaterais**, **contexto**, entre outros, dentro de componentes funcionais. Neste capítulo, vamos explorar os três hooks fundamentais: **useState**, **useEffect** e **useContext**, com exemplos práticos e explicações didáticas.

6.1 useState

O hook `useState` permite que componentes funcionais mantenham e atualizem valores ao longo do tempo.

a) Criar componente **ContadorSimples** em **src/components/ContadorSimples.jsx**

```
1 import { useState } from 'react';
2 function ContadorSimples() {
3   const [valor, setValor] = useState(0);
4   return (
5     <div>
6       <p>Contador: {valor}</p>
7       <button onClick={() => setValor(valor + 1)}>Incrementar</button>
8     </div>
9   );
10 }
11 export default ContadorSimples;
```

Figura 59 - Componente hook `useState`
Fonte: Elaborado pelo autor (2025)

b) Testar no **App.jsx**

```
1 import ContadorSimples from './components/ContadorSimples';
2 function App() {
3   return <ContadorSimples />;
4 }
5 export default App;
```

Figura 60 - **App.jsx** – hook `useState`
Fonte: Elaborado pelo autor (2025)

6.2 useEffect

O `useEffect` é um hook do React utilizado para **executar efeitos colaterais** em componentes funcionais. Efeitos colaterais são ações que **ocorrem fora do fluxo natural de renderização** da interface, mas que são necessárias durante o ciclo de vida do componente.

6.2.1 Para que serve o `useEffect`?

Você utilizará o `useEffect` sempre que precisar realizar tarefas como:

- Buscar dados em **APIs externas** assim que o componente for exibido;
- Atualizar o **título da aba do navegador** com base em alguma interação;
- Registrar ou remover eventos de teclado ou clique;
- Controlar intervalos e timeouts;
- Sincronizar dados com `localStorage`, banco local ou bibliotecas externas.

6.2.2 Quando o `useEffect` é executado?

O `useEffect` é executado após a renderização do componente. Com ele, é possível controlar exatamente quando e com que frequência a função será executada:

- **Montagem:** com `[]` como segundo parâmetro, o efeito é executado apenas uma vez (semelhante ao `componentDidMount`);
- **Atualizações:** passando as variáveis no array de dependências, o efeito roda sempre que uma delas mudar;
- **Desmontagem:** usando uma função de retorno, é possível realizar limpezas (como encerrar timers ou desconectar sockets).

6.2.3 Exemplo prático

a) Criar componente **Relógio** em **src/components/Relogio.jsx**

```
1 import { useState, useEffect } from 'react';
2 function Relogio() {
3   const [hora, setHora] = useState(new Date());
4   useEffect(() => {
5     const timer = setInterval(() => {
6       setHora(new Date());
7     }, 1000);
8     return () => clearInterval(timer);
9   }, []);
10  return <h2>{hora.toLocaleTimeString()}</h2>;
11 }
12 export default Relogio;
```

Figura 61 - Componente Relógio
Fonte: Elaborado pelo autor (2025)

b) Testar no **App.jsx**

```
1 import Relogio from './components/Relogio';
2 function App() {
3   return <Relogio />;
4 }
5 export default App;
```

Figura 62 - App.jsx – Componente Relógio
Fonte: Elaborado pelo autor (2025)

Explicação: o array vazio [] no useEffect indica que o efeito só deve rodar uma vez após o componente ser montado. A função de retorno faz a limpeza do intervalo para evitar vazamentos de memória.

6.3 useContext

O React oferece várias formas de compartilhar dados entre componentes. A forma mais comum é por meio de **props**, onde um componente pai passa dados para seus filhos através de atributos. No entanto, quando os dados precisam ser acessados por muitos componentes em diferentes níveis da aplicação, **passar props manualmente** em cada etapa pode se tornar **ineficiente, repetitivo e difícil de manter**.

Para solucionar esse problema, o React disponibiliza a **API de Contexto**, que funciona como um **canal de comunicação global** entre os componentes.

6.3.1 O que é o useContext?

O useContext é um **hook** que permite a qualquer componente acessar um contexto previamente definido, **sem a necessidade de repassar props** manualmente ao longo da hierarquia de componentes.

Esse hook é ideal para **compartilhar informações globais** como:

- Dados do usuário logado;
- Idioma selecionado;
- Tema da aplicação (claro/escuro);
- Configurações do sistema;
- Permissões ou autenticação.

6.3.2 Como funciona o uso de contexto?

1. **Criação do contexto:** define-se uma estrutura (como uma variável global) que será compartilhada com os componentes interessados.
2. **Provedor de contexto (Context.Provider):** envolve os componentes que terão acesso ao dado, definindo o valor que será compartilhado.
3. **Consumidor do contexto com useContext:** qualquer componente filho pode acessar o valor sem precisar receber via props.

6.3.3 Quando usar useContext?

Use useContext quando:

- Os dados forem utilizados por vários componentes em diferentes níveis da hierarquia;
- Você quiser centralizar e organizar o estado global da aplicação;

- Quiser **evitar o “prop drilling”**, que é o processo de passar props através de vários níveis intermediários que não precisam desses dados.

6.3.4 Limites do useContext

Apesar de ser muito útil, o useContext **não substitui ferramentas como Redux ou Zustand** quando há necessidade de gerenciar estados mais complexos, com múltiplas alterações simultâneas ou efeitos colaterais interdependentes. Ele também **não gerencia atualizações de forma eficiente em larga escala**, pois qualquer alteração no contexto provoca a re-renderização de todos os consumidores.

O hook useContext permite compartilhar dados entre componentes sem a necessidade de repassar via props manualmente em cada nível da árvore.

a) Criar contexto em **src/context/usuariocontext.jsx**

```
1 import { createContext } from 'react';
2 export const UsuarioContext = createContext(null);
```

Figura 63 - Limites do useContext
Fonte: Elaborado pelo autor (2025)

b) Criar componente **PerfilUsuario** em **src/components/PerfilUsuario.jsx**

```
1 import { useContext } from 'react';
2 import { UsuarioContext } from '../context/UsuarioContext';
3 function PerfilUsuario() {
4   const usuario = useContext(UsuarioContext);
5   return <p>Usuário logado: {usuario.nome}</p>;
6 }
7 export default PerfilUsuario;
```

Figura 64 - Componente PerfilUsuario
Fonte: Elaborado pelo autor (2025)

c) Atualizar **App.jsx** para fornecer o contexto.

Frontend Mágico: Interfaces que Encantam no Mobile

```
1 import { UsuarioContext } from './context/UsuarioContext';
2 import PerfilUsuario from './components/PerfilUsuario';
3 function App() {
4   const usuario = { nome: 'Fernanda', email: 'fer@exemplo.com' };
5   return (
6     <UsuarioContext.Provider value={usuario}>
7       <PerfilUsuario />
8     </UsuarioContext.Provider>
9   );
10 }
11 export default App;
```

Figura 65 - App.jsx – PerfilUsuario
Fonte: Elaborado pelo autor (2025)

Dica: utilize o **Context** quando precisar compartilhar informações como usuário logado, tema da aplicação ou configurações globais entre diversos componentes.

REFERÊNCIAS

DEEPCOOMER. The Evolution of JavaScript: From Scripting Language to Modern Powerhouse. **Medium**, 2 de setembro de 2023. Disponível em: <<https://medium.com/@deepcoomer45/the-evolution-of-javascript-from-scripting-language-to-modern-powerhouse-adee5b7cdd09>>. Acesso em: 16 maio 2025.

PONTES, Guilherme. **Progressive web apps**: construa aplicações progressivas com React. São Paulo: Casa do Código, 2018.

THE build tool for the web. **VITE**, 2025. Disponível em: <<https://vite.dev/>>. Acesso em: 12 maio 2025.